

암호구현

256비트 순열 복원과 최적화

2026년 7월

결과 요약

복원한 1라운드 순열은 다음과 같다.

rotate → XOR → reverse all 32 bytes → add,

워드별 rotation은 다음과 같다.

rot = {43, 7, 29, 14}.

최종 구현은 제공된 1,000개 1라운드 벡터, 제공된 20라운드 벡터, 그리고 100,000개의 결정론적 랜덤 차분 테스트를 모두 통과한다. 최적화의 핵심은 두 가지다: (1) 범용 byte map을 4개의 64비트 byte swap으로 대체하고, (2) 라운드를 2개씩 묶어 word reversal을 상쇄시켜 4개의 독립적인 scalar dependency chain을 노출한 뒤, 10개 쌍을 모두 unroll하고 local BMI2 target으로 GCC가 non-destructive constant rotate를 쓰게 한다. 같은 소스의 적응형 score build(전역 BMI2 + 큰 inline threshold로 public wrapper를 timing loop에 통합)는 두 차례 반복 측정에서 기본 build 대비 paired speedup 1.116x와 1.118x를 얻었다.

- **Target-only 후보:** lane-wise AVX2 구현은 정확한 score loop를 322개 scalar instruction에서 122개 vector instruction으로 줄이지만, AMD affinity 결과가 코어마다 엇갈려 scalar를 능가하는지 판정할 수 없다. 실제 Core Ultra 7 255H 측정 전까지는 scalar 소스가 제출 incumbent로 남는다.
- **검증 강화:** Schema 5는 매 측정 프로세스 후 정확한 최종 상태를 검증해, 예전 gate가 2.253배 빠르다고 잘못 보고했던 의도적으로 절반만 도는 loop 같은 오류를 걸러낸다.
- register/encoding 조정, timing 안정성 강화, 3-instruction ISA 합성 탐색을 포함한 11차에 걸친 세부 스크리닝은 § 5에 정리했다. 어떤 시도도 AMD 호스트 측정만으로는 scalar 제출본을 대체하지 못했다.

현재 제출 상태

- 정확성과 최종 scalar 제출 소스는 확정되었다. 제출 파일은 submissions/02/contest.c이다.
- 권장 score build는 다음과 같다.

```
gcc -O3 -Wall -Wextra -mbmi2 -finline-limit=2000 contest.c
```

- 성능 우승자는 Intel Core Ultra 7 255H의 P, E, LP-E core를 직접 측정으로 구분하기 전까지는 잠정적이다. 그때까지는 scalar incumbent가 제출된 구현으로 남는다.

- 최우선 A/B 후보는 lane-wise AVX2다. counted block2, block3+tail1, block5 front end가 그 다음 후보다. 교체하려면 요청된 모든 core type에서 서로 다른 시점에 수행한 두 세션 모두 일관되게 개선되어야 한다.
- 본 보고서의 정확한 mnemonic count와 byte size는 각 표에 명시한 컴파일러 버전이 만든 binary를 기술한 것이다. GCC 13.3.0과 다른 컴파일러는 동일한 C를 다르게 배치할 수 있으므로, 이 수치들은 정확성 조건이나 target 성능 순위가 아니라 audit 기록이다.

1 Permutation 구조 분석 과정

모든 구성 연산은 invertible하다. 덧셈은 2^{64} modulo 뺄셈으로, XOR은 self-inverse로, 그리고 map shuffle_map[i] = 31-i도 self-inverse로 역산된다. 제공된 모든 input/output 쌍에 대해 다음을 계산했다.

```
t0 = output - constants1 // modulo  $2^{64}$  per word
t1 = reverse_all_32_bytes(t0)
t2 = t1 XOR constants2
```

2 rot 값 도출 과정

그리고 허용된 각 $r \in \{1, \dots, 63\}$ 에 대해 다음을 검증했다.

```
ROTL64(input[i], r) == t2[i].
```

복원 스크립트는 하나의 (변질됐을 수 있는) word만 신뢰하는 대신 1,000개 벡터 전체에서 얻은 후보 집합의 교집합을 구한다. 각 word는 후보가 하나씩 남는다. 43, 7, 29, 14. forward evaluation은 제공된 모든 벡터에서 앞서 말한 연산 순서를 확인해 준다.

3 구현 방법 상세 설명

direct reference는 복원한 순서대로 제공된 연산을 호출한다.

```
rotate_words_left_64wise(state, rot);
xor_constants_256wise(state, constants2);
shuffle_bytes_256(state, shuffle_map);
add_constants_64wise(state, constants1);
```

고정된 map은 임의의 순열이 아니라 완전한 byte reversal이다.

```
reverse32(x0,x1,x2,x3) =
  (BSWAP64(x3), BSWAP64(x2), BSWAP64(x1), BSWAP64(x0)).
```

따라서 인덱스로 지정한 32개 byte 복사와 두 개의 임시 배열이 4개의 __builtin_bswap64 연산으로 바뀐다. 하나로 특수화한 라운드는 다음과 같다.

```
y0 = BSWAP64(ROTL64(x3,14) ^ C2[3]) + C1[0]
y1 = BSWAP64(ROTL64(x2,29) ^ C2[2]) + C1[1]
y2 = BSWAP64(ROTL64(x1, 7) ^ C2[1]) + C1[2]
y3 = BSWAP64(ROTL64(x0,43) ^ C2[0]) + C1[3]
```

고정된 shift count 상수 덕분에 immediate-count rotate instruction을 쓸 수 있다. 최적화 이전 제출본은 제공된 loop에서 1라운드짜리 state-writing helper를 20번 호출했다. register-resident 20라운드 loop를 실험용 baseline으로 사용했다.

3.1 2라운드 결합

한 라운드는 4개 word 위치를 뒤집는다. 두 번 뒤집으면 원래 위치로 돌아오므로, 2라운드마다의 출력은 같은 인덱스의 word에만 의존한다. source word j 에 연관된 rotate, XOR, byte-swap, destination addition을 T_j 라 하면 2라운드는 다음과 같다.

```
x0 <- T3(T0(x0)) x1 <- T2(T1(x1))
x2 <- T1(T2(x2)) x3 <- T0(T3(x3)).
```

이 형태는 라운드마다의 word 이동 없이 4개의 독립적인 chain을 드러낸다. helper는 이 block을 10번 명시적으로 전개한다. GCC의 local target("bmi2") attribute는 condition flag를 갱신하지 않는 non-destructive rotate인 RORX를 허용하며, contest 파일의 나머지 부분은 여전히 제공된 기본 명령으로 컴파일된다.

helper의 attribute는 __BMI2__에 조건부다. 일반 명령으로는 정렬된 local-target noline helper로 남는다. -mbmi2 -finline-limit=2000에서는 caller와 callee의 target option이 일치해 core가 public 함수 안으로 강제 편입되고, GCC는 그 함수를 main에도 inline한다. 이는 추가 flag가 없는 호환 경로를 유지하면서, 문제 규정의 optimization-flag 규칙이 허용하는 더 빠른 경로를 가능하게 한다.

문제 규정은 외부 helper 함수를 허용하고 제공된 20라운드 loop 내부의 수정을 제한한다. 허용된 그 한 줄은 첫 반복에서 helper를 호출해 20개 라운드를 모두 처리하고, 제공된 loop counter를 19로 설정한다. helper는 어떤 입력 state에도 동일한 순열을 적용하며, benchmark 입력이나 출력을 하드코딩하지 않는다.

4 테스트 벡터 검증 결과

다음 항목들은 모두 통과한다.

- 제공된 1,000개 1라운드 input/output 쌍 전부;
- 제공된 20라운드 쌍;
- 1라운드와 20라운드에서 100,000개의 결정론적 random state와 random add/XOR constant에 대해 generic reference와 비교한 각 후보의 public 함수;
- 보존된 이전 버전과 최종 contest.c 파일 각각의 별도 complete binary; 그리고
- 측정된 모든 후보 process에 대한 schema-5 repeated-call과 nonce로 유도한 대체-count 최종 state.

```
one_round_vectors=1000
twenty_round_vector=PASS
random_differential_cases=100000
selftest=PASS
```

```
one-round testvector verification: OK (1000 pairs checked)
20-round testvector verification: OK
```

5 성능 최적화 기법

실험 harness는 각 후보를 별도 binary로 빌드해, 크게 unroll된 함수들이 서로의 instruction-cache layout을 바꾸지 않게 한다.

이 절 전체에서 별도 언급이 없는 한 다음 두 조건이 성립한다. **정확성:** 모든 후보는 제공된 벡터와, 1라운드/20라운드에서의 100,000개 random-state/random-constant case, 그리고 timed loop에

대한 정확한 audit을 통과한다. **하드웨어:** timing 수치는 target인 Intel Core Ultra 7 255H가 아니라 AMD host에서 얻은 것이므로, AMD speedup은 255H 결과가 아니라 code shape에 대한 진단용/target-only 증거이며, 오직 직접적인 255H A/B 테스트만이 후보를 승격시킨다. 이후의 소절은 차이점과 이 표시만 “target-only”/“diagnostic”로 언급한다.

GCC 12.2.0을 사용한 AMD EPYC 7B12 VM에서 -O3 -march=native, CPU 2, 15회 반복 sample로 screening한 결과는 다음과 같다.

후보	Median ns	MAD ns	vs. pre-opt.
Preserved pre-optimization form	41.894	2.834	1.000x
Register loop	40.655	3.083	1.031x
Paired AVX2 loop	39.718	0.131	1.055x
Paired scalar loop	37.751	2.649	1.110x
Paired AVX2 full unroll	39.417	0.086	1.063x
Paired scalar full unroll	36.824	2.617	1.138x
Paired scalar unroll + BMI2	36.249	3.195	1.156x
One-state AVX2	73.630	0.266	0.569x

one-state AVX2 버전이 느린 이유는 의존적인 각 라운드가 가변 left/right shift, OR, XOR, byte shuffle, half permutation, addition을 요구하기 때문이다. 반면 4개의 scalar word는 더 짧은 4개의 독립 chain을 제공한다. transposed four-state AVX2 버전은 throughput을 높이지만, 공식 API는 하나의 state를 순차적으로 갱신하므로 그런 batching을 쓸 수 없다. paired AVX2 loop는 이 host에서 안정적이었으나 scalar BMI2보다 느렸고, non-BMI2 inline unroll은 contest-shaped 테스트에서 BMI2와 거의 동률이어서 둘 다 유용한 target-machine 대조군으로 남는다.

5.1 Frontend와 cache 심층 검토

18-후보 스왑은 unroll factor, 16/32/64/128바이트 alignment, literal constant, inlining, wrapper layout, SIMD, chain scheduling을 바꿔가며 시험했다. 제출된 helper는 1,267바이트이고 hot-body spill이 없으며 4개 state word와 8개 constant를 register에 유지한다. 4개의 독립적인 RORX/XOR/BSWAP/ADD chain이 단순 80-cycle dependency depth에 근접해 실행되어, 남은 차이는 주로 frontend와 code layout 때문이다.

715바이트짜리 절반만 unroll한 loop는 한 paired session에서 1.0867x를 기록했지만 곧바로 재측정하면 0.9712x, -march=native에서는 0.9214x였다. alignment 변경, embedded constant, forced inline은 중립이거나 더 느렸고, 직렬화된 chain은 0.7305x에 그쳤다. cross-call inline flag 아래 full/pair-loop/half-unroll의 median은 각각 37.279/38.618/38.727 ns로, full unroll이 여전히 산술 core로 남는다.

5.2 Cross-call inlining

-mbmi2만 지정하면 GCC는 private core를 public permute_20rounds에 합치지만, main의 timing loop는 여전히 그 함수를 호출한다. -finline-limit=2000을 추가하면 이 두 번째 경계가 사라진다. assembly 검사 결과 timed loop 안에는 core-external call, push/pop, state/constant memory operand가 전혀 없었고, 12개 값 모두가 outer iteration 내내 register에 남았다. default/-mbmi2/full-inline ablation은 paired 1.000x/0.999x/1.124x를 측정해, public-to-main 경계 제거가 이득의 원인임을 확인했다. GCC 12는 inline limit 700과 2000에서 byte-identical binary를 생성했고, digest로 고정한 공식 GCC 13.3.0 build도 target 컴파일러에서 두 complete binary가 byte-identical임을 확인했다. 명시적인 32/64/128바이트 alignment는 inline된 loop 진입점을 옮겼지만 paired 1.006x/0.991x/0.990x를 측정했고, 모든 bootstrap 구간이 1에 닿거나 가로질렀으므로 기본 alignment를 유지한다. 이전의 generic IRA/native 대조군도 confidence gate를 통과하지 못했다. 따라서 권장 score 명령은 real-255H 확인 없이는 두 추가 flag 중 어느 것도 유지하지 않는다.

저장소의 assembly audit은 마지막 두 clobber() 호출 사이의 timed backedge를 찾아낸다. 두 inline limit 모두 동일하게 정규화된 1,216바이트, 322-instruction loop를 만든다. RORX, XOR, BSWAP, ADD/LEA가 각각 80개씩이며 call, push/pop, state/constant memory operand는 없다. portable forced-inline 대조군은 전체 text를 292바이트 줄이고 loop를 1,060바이트로 줄였지만, destructive ROL/ROR가 move를 하나 추가해 0.985x(95% CI 0.958–1.002x)에 그쳤다. 더 작은 코드가 BMI2를 이기지 못했다.

5.3 두 번째 파의 대안들

byte swap은 XOR와 commute하며 $(B(u \oplus K) = B(u) \oplus B(K))$, 2^{63} 과의 XOR은 2^{64} modulo에서 2^{63} 덧셈과 같다. 이를 이용한 미리-swap한 constant와 1비트 함수는 instruction을 줄이지 못했고, 1.095x–1.116x로 보였던 별도-process 이득은 같은-process AB/BA 측정(1.0146x, 0.9943x)에서 사라졌으며, 손으로 작성한 inline assembly도 0.9975x에 그쳤다.

opposite-phase 변환을 결합(conjugate)해 2-lane SIMD chain 2개를 쓰면 core가 약 320개에서 240개 instruction으로 줄지만, 4개의 독립적 scalar chain이 더 긴 2개의 vector chain이 되고 각 rotate가 shift/shift/OR를 요구해 0.845x–0.848x에 그쳤다.

정확한 64비트 단항 함수 table은 함수당 128 EiB, 두 conjugacy class를 합치면 256 EiB가 필요하며, influence 및 mixed-derivative 검사도 byte-independent XOR/add table 분해를 기각했다. 64개의 input/output-byte edge가 모두 나타났고, 1,000개의 mixed XOR derivative가 모두 0이 아니었으며, 제한된 16차원 ANF는 64개 출력 비트 중 29개에서 차수 16에 도달했다. 이 관측들은 일반적인 circuit lower bound를 증명하지는 않지만, 제안된 small-table 형태를 직접 반증한다. compiler scheduling, target clone, PGO/LTO, 수동 BMI2 scheduling, cross-call batching도 이식 가능한 유효 우승자를 내지 못했으며, batching은 순차적인 state recurrence와 호환되지 않는다.

실제 constant에 대해 전수 low-bit projection도 XOR를 ADD 너머로 이동시키는 것을 기각한다. 방정식 $(x \oplus C) + A = (x + B) \oplus E$ 는 네 변환에 대해 폭 5, 3, 10, 4에서 해가 없다. bit 63을 ADD로 접는 것은 두 constant에서는 성립하지만 0이 아닌 XOR mask를 남긴다. 4개의 2라운드 선형 bit permutation은 rotate 하나로도, rotate/byte-swap 결합 하나로도 일치하지 않는다. 정확한 대안들은 post-BSWAP XOR에서 0.980x, top-bit folding에서 0.983x, arithmetic XOR에서 0.620x, 32비트 half에서 0.251x를 기록했다.

마지막 complete-binary 스크리닝은 120개의 GCC/link 조합을 컴파일해 26개의 hot-loop stream을 찾았다. LTO, whole-program, semantic-interposition, section-GC 옵션은 hot loop를 byte-identical하게 남겼다. 1.0217x로 보였던 IRA 승리는 4개의 통제된 offset(0.985–0.995x, 모든 구간이 1을 가로지름)에서 사라졌고, PGO/scheduling도 패했다. GCC 13.3의 -mtune=alderlake는 AMD codegen 실행에서 1.0148x였으나 (target-only) real-255H A/B 후보로만 남는다.

5.4 정확한 GCC 13.3과 4파 탐색

공식 gcc:13.3.0 이미지를 digest로 고정해 수정하지 않은 최종 소스를 컴파일했다. 저장된 재현 기록에는 컴파일러, Binutils와 소스 hash, 직접 correctness 결과, 정확한 timed-loop audit이 담겨 있다.

빌드	.text	측정 loop	감사/결과
Default -O3	5,972 B	31 B / 6 insn	helper 호출 1회
BMI2 inline 700/2000	7,246 B	1,216 B / 322 insn	동일 binary
Alder tune	7,246 B	1,216 B / 322 insn	다른 schedule
Alder tune + IRA	7,199 B	1,210 B / 322 insn	세 번째 schedule

모든 inline loop는 core-external call, stack 연산, hot memory operand가 0개이며, 모든 build는 표준 correctness gate를 통과한다. GCC 13.3은 -mtune=alderlake, raptorlake, meteorlake에 대해 동일한 complete binary를 내며, arrowlake tune 이름은 제공하지 않는다.

34개 configuration의 scheduler/allocator 스크리닝은 32개의 유효한 build를 냈지만 loop stream은 8개뿐이었다. LLVM-MCA 16의 근사 hybrid model은 generic 125.06cycle, Alder 123.62cycle, Alder+IRA 121.06cycle을 예측한다. AMD host에서 더 엄격한 5,000,000회 호출, 6회 warm-up, 40-sample 비교는 CPU 0에서 Alder-to-Alder+IRA paired 1.011x (CI 1.001–1.015x), CPU 4에서 1.008x(0.998–1.025x)를 주었다. 구간이 서로 일관되지 않아 승격은 불가능하지만(target-only), 세 번째 stream은 high-priority 255H 후보다.

BSWAP 뒤나 rotate 앞으로 XOR를 옮기는 것은 임의 constant에 대해 정확하지만 320개 core 연산을 그대로 유지한다. CPU 0/4에서의 과거 schema-4, 24-sample 측정은 post-BSWAP에서 0.959x/0.963x, pre-rotate에서 0.965x/0.960x를 기록했고, 네 구간 모두 1 미만이었다. constant를 memory에 강제하면 160개의 hot memory operand가 생기고 loop가 2,015바이트로 커지며, 두 번의 21-sample 측정은 0.979x(CI 0.964–0.992x)와 0.988x(0.969–1.002x) 였다. literal 특수화는 random-constant 규정을 위반하며 그럼에도 연산을 줄이지 못한다. 결정론적 검사에 따르면 원본, byte-swap한, inverse-rotate한, top-bit-fold한 mask 모두 sign-extended imm32에도, 완전히 흡수 가능한 유일한 XOR mask인 $\{0, 2^{63}\}$ 에도 맞지 않는다.

5.5 5파 소스 순서와 backend schedule

2라운드 block 안의 네 할당은 서로 독립적이다. 그래서 digest로 고정한 GCC 13.3 스크리닝은 24가지 소스 순서 전부를 generic, Alder, Alder+IRA profile로 컴파일했다. 순서 2, 1, 0, 3은 LLVM-MCA의 근사 generic/Alder 결과를 125.06/123.62에서 121.06cycle로 줄였고, Alder+IRA는 121.06 그대로였다. generic/Alder loop는 322 instruction과 320개 core 연산을 모두 유지하면서 1,216바이트에서 1,211바이트로 줄었다. 세 profile 모두 call, push/pop, spill, hot memory operand가 없으며 표준 correctness gate도 통과한다.

별도의 106-flag scheduler/layout 스크리닝은 9개의 loop stream을 냈다. 106개 build, 3개의 source-order cross-product, 9개의 최종 후보 변형 모두 정확한 assembly audit과 표준 correctness gate를 통과한다. 근사 Alder model에서 -fselective-scheduling2와 -fno-schedule-insns2는 Alder+IRA의 121.06cycle 대비 각각 120.06, 120.07cycle을, critical path heuristic을 끄면 120.14를 주었다. 64바이트 alignment와 LTO는 배치는 바꾸었지만 121.06-cycle loop stream 자체는 바꾸지 못했다. 소스 순서와 앞의 두 flag를 결합해도 static 이득이 누적되지 않았다. AMD 진단은 CPU마다 달랐다 (target-only). 이 build들은 255H screen/confirm manifest에만 추가되었을 뿐 incumbent는 그대로다.

5.6 6파 대수적 탐색과 lane-wise SIMD 탐색

더 작은 scalar pair block을 다시 확인했다. 1/2/5/10-block loop는 각각 143/278/646/1,211바이트였다. compact loop의 초기 21-sample 결과는 1.035x(CI 1.007–1.043)였지만, 6회 warm-up, 6,000,000회 호출, 32-sample 확인 측정에서는 1.0004x(0.9855–1.0176)에 그쳐 이 재현 실패로 승격이 막혔다.

더 짧은 scalar identity 탐색에서 한 stage를 BSWAP64($\text{ROL64}(x, r) \text{ xor } k$) + a 로 모델링하면, Z3는 서로 다른 각 stage에 대해 길이-3 rotate/XOR/byte-swap/add template 64개 전부가 UNSAT임을 확인했다. 기존 8-연산 pair에서 연산을 하나씩 지우고 constant를 재합성한 경우도 32개 모두 UNSAT이었다. 전수 선형 탐색은 길이 3 이하의 4,223개 구문, 632개의 서로 다른 bit permutation을 열거했지만 어느 pair와도 일치하지 않았다. 이는 이 grammar에 대한 local lower bound일 뿐, 임의의 machine program에 대한 전역 lower bound가 아니다.

새 알고리즘은 대신 독립적인 4개의 2라운드 chain을 4개의 YMM lane에 배치한다. 12-variant register-residency 스크리닝은 마지막 두 constant load를 제거하는 identity-helper 형태를 찾았다. 정확한 GCC 13.3은 122 instruction, 579바이트를 내며 hot memory operand가 없다: VPSLLVQ, VPSRLVQ, VPOR, VPXOR, VPSHUF, VPADDQ가 정확히 20개씩, 그리고 SUB/JNE이 있다. 이 정리 작업 자체는 이전의 124-instruction vector stream과 1.0013x(0.9983–1.0039)로 동률이었다.

AMD affinity	scalar ns	AVX2 ns	paired	95% CI
CPU 1	46.927	36.690	1.275x	1.260–1.292
CPU 2	34.354	36.569	0.940x	0.923–0.950
CPU 3	45.938	36.724	1.248x	1.222–1.269

각 행은 6회의 폐기된 warm-up, 32개의 균형 sample, sample당 3,000,000회 호출, 직접적인 100,000-case random-state/random-constant gate를 사용한다. CPU 2는 CPU 1/3의 순서를 뒤집어, 이 VM조차 255H 측정을 대신할 수 없음을 보여준다. SSE2는 781 instruction, 4,064바이트, 약 0.40x로 기각되었다. 공식 Core Ultra 7 255H 제품 사양은 vector extension으로 AVX2를 명시하므로, AVX-512 후보는 지원 대상 target 범위 밖이라 시도하지 않았다. 47개의 추가 exact-GCC13과 53개의 Clang 21 target/scheduler 조합에서도 더 나은 static stream을 찾지 못했다. Clang의 spill-free 1,208바이트 Arrow Lake stream은 근사 Alder model에서 133.79점을 받았고, LLVM-MCA 16에는 Lion Cove나 Skymont model이 없다. 따라서 scalar build가 여전히 incumbent이고, 122-instruction AVX2 build가 선두 target-only 후보다.

5.7 7파 split SIMD, code generation, timing 통제

YMM 구현은 instruction 수를 최소화하지만 하나의 vector dependency chain을 유지한다. 그래서 4가지 two-XMM 배치(contiguous lane, reversal-orbit pair, serialized group, explicit lane recomputation)를 시험했다. 모두 정확한 GCC 13.3 audit과 표준 correctness gate를 통과했지만 분명한 static reject였다.

설계	Instruction	Bytes	Hot memory	Alder/YMM
Contiguous XMM	242	1,349	50	2.23x
Orbit pairs	248	1,383	50	2.35x
Serialized	250	1,303	30	1.78x
Recompute lanes	288	1,509	30	1.91x

교차-architecture static ratio가 가장 좋은 경우조차 현재 YMM loop의 1.36x였다. 중복된 128비트 연산과 불변 spill은 시끄러운 host 측정을 정당화할 근거가 없으므로, 이 소스들은 재현 가능한 부정적 실험으로 남으며 255H manifest에는 포함되지 않는다.

두 번째 스크리닝은 113개의 exact GCC 13.3과 Clang 21 target, scheduler, register-allocation, alignment, source-expression build를 컴파일했다. 100개의 complete timed loop와 모든 Pareto/source 대조군이 정확한 audit과 표준 correctness gate를 통과했다. GCC의 -fira-region=one은 122 instruction, hot memory, 두 model의 cycle 추정치를 바꾸지 않으면서 579바이트를 569바이트로 줄였다. Clang은 같은 instruction/model 결과로 548바이트를 냈지만 심사 컴파일러는 아니다. 상보적인 rotate shift는 bit 단위로 겹치지 않으므로 VPOR을 VPXOR로 바꾸는 것이 정확함을 두 컴파일러 모두 확인했지만, mnemonic 구성만 바뀌었다. source만으로 작성한 inline assembly RORX 대조군도 일반 gcc -O3에서는 public call을 남겼으므로, score inline flag는 여전히 필요하다. 살아있는 모든 YMM 값을 고정하면 encoding이 563바이트로 줄었지만, GCC가 hot reload 1개와 reverse permutation 1개를 추가해 124 instruction이 되어, correctness는 통과했지만 성능 audit에서 탈락했다.

timing-stability 도구는 4-KiB로 정렬한 두 개의 same-process runner를 AB/BA 교대 순서로 실행하며 분리된 complete process와 비교했다. 각 sample은 wall time, thread CPU time, 직렬화한 RDTSCP, CPU와 TSC_AUX, context switch, SMT-sibling load를 기록한다. CPU 2에서, 위와 같은 warm-up/sample/call protocol 아래 다음 결과를 얻었다.

통제	Scalar/AVX2 paired	95% CI
Process isolated	0.8768x	0.859–0.891
Same process, page aligned	0.9225x	0.8845–0.9290

wall, thread-CPU, invariant-TSC 비율은 0.000003 이내로 일치했고, migration이나 TSC_AUX 변화도 없었다. CPU 1/2/3은 byte-identical한 scalar와 AVX2 binary, 정규화된 loop를 사용했다. AVX2 median은 0.78%만 변한 반면 scalar median은 45.89% 변했다. 따라서 이 역전 현상은 code generation이나 timer 선택이 아니라 공유 host의 scalar throughput 문제로 국한된다. SMT load가 그럴듯한 원인이지만 APERF/MPERF, cpufreq, performance counter를 사용할 수 없어 인과적인 frequency/SMT 주장은 할 수 없으며, 이 진단은 255H 성능을 예측하지 않는다.

5.8 8파 register allocation과 target 증거

563바이트짜리 기각된 stream은 low-register encoding이 유용하지만 살아있는 모든 값을 고정하면 pressure가 너무 커짐을 보여주었다. 그래서 제한된 10-case 스크리닝으로 allocator 자유도, scratch register 1개 대 2개, 부분 고정, encoding-aware low/high 배치를 바꿔가며 시험했다. register map을 brute-force하지는 않은 채, 모든 후보를 정확한 GCC 13.3으로 컴파일해 표준 correctness gate로 검증했다.

승리한 형태는 변화하는 state만 ymm0에 고정한다. 각 rotate마다 먼저 right shift를 scratch 하나에 쓰고, state를 destructive하게 left-shift한 뒤 scratch를 OR로 되돌린다. GCC가 나머지 constant와 제어를 모두 할당한다. 그 결과 122 instruction, hot memory 0개, 동일한 100.03/180.03 Alder/Zen 대리 cycle을 유지하면서 loop를 579바이트에서 **569바이트**로 줄였다. 이는 크기가 같은 -fira-region=one build와는 별개의 정규화된 stream이다. 다른 8가지 assembly 할당은 124 instruction과 한두 개의 hot load를 냈다. 채택된 소스는 제공된 기본 GCC 명령과 공식 벡터 harness도 통과한다.

두 번째 대수적 실험은 두 reversal orbit을 phase-stagger했다. $T_j(x) = \text{BSWAP}(\text{ROL}(x, r_j) \text{ xor } k_j) + a_{3-j}$ 로 두고 $[x_0, T_3(x_3)]$ 와 $[x_1, T_2(x_2)]$ 를 묶어, orbit마다 19개의 immediate-rotate stage를 공유하고 lane epilogue를 하나 적용한다. 이는 임의 constant에 대해 정확하고 hot memory가 없지만, stream이 257 instruction과 1,253바이트로 중복된다. LLVM-MCA는 Alder에서 100.03 대비 116.04cycle을 추정했지만, Zen 2 대리 지표에서는 180.03 대비 143.05였으므로 교차-architecture 결과는 실측이 필요했다.

AMD affinity	569-byte assembly speedup	Phase-stagger speedup
CPU 1	0.999x (0.998-1.002)	0.758x (0.754-0.764)
CPU 3	1.000x (0.998-1.002)	0.756x (0.751-0.764)

각 행은 위와 같은 warm-up/sample/call protocol에 더해 직접 100,000-case 검증과 정확한 측정-binary audit을 사용한다. 569바이트 소스는 두 affinity 모두에서 통계적으로 동물이다. phase staggering은 throughput이 약 24% 낮고(같은 작업에 대해 실행시간이 약 32% 더 길고), 유리해 보였던 Zen 2 scheduling 대리 지표와 정면으로 배치된다. 이는 재현 가능한 부정적 실험으로 남으며 target manifest에는 없다. 첫 시도에서는 상대 경로의 quoted include가 있는 소스를 복사하며 원래 include context가 사라지는 benchmark staging 버그도 드러났다. driver는 이제 -iquote로 두 build 모두에 원래 소스 디렉터리를 제공하고 이를 해당 historical schema-4 파일에 기록하며, 위 두 측정은 모두 깨끗하게 재실행한 결과다.

추가 exact-GCC13 backend 스크리닝은 63가지의 새로운 preferred-width, load/store 분할, VEX/vzeroupper, move/store cap, cost-model, partial/AVX tune 조합에 2개의 기준선을 더해 시험했다. 65개 audit 전부 통과했지만, 모든 결과가 122 instruction, 579바이트, 변함없는 대리 cycle의 이미 알려진 두 generic/Alder 정규화 stream으로 수렴했다. 배치까지 동일하지는 않아, (stream hash, loop-start mod 64) 기준으로 8개 class가 나왔으며, generic offset은 0/8/24/40/48, Alder offset은 8/16/48이었다. 각 class의 대표 하나씩이 표준 correctness gate를 통과했다. LLVM-MCA는 이 frontend 배치 효과를 모델링하지 않으므로, 7개의 non-baseline 대표들은 이 제한된 flag 결과로 기각되는 대신 target-only로 남는다.

instruction sensitivity model은 Intel의 공식 Skymont와 Crestmont 패키지를 archive와 payload hash로 고정한다. Skymont 다운로드에는 Xeon 6 E-core용으로 명명되어 있어, 그 값을 client 255H로 옮기는 것은 여전히 조건부다. 공식 Intel 페이지 두 곳도 서로 충돌한다. Arrow Lake

PerfMon은 LP-E를 Crestmont라 부르지만, 255H 전용 ECI 페이지는 두 LP-E core를 추가 Skymont core라 부른다. 따라서 Crestmont는 PerfMon-mapping sensitivity case로만 유지된다. 선택한 AVX2 행들은 두 표 모두에서 순차적 YMM chain에 100-cycle latency 경로를 준다. scalar의 80-cycle 시나리오는 조건부다. 두 표 모두 정확한 64비트 RORX와 6개의 LEA 행을 생략하고, 2026-07-23에 고정한 Intel 카탈로그에는 공식 공개된 Lion Cove per-instruction table이 없었다. 고립된 reciprocal throughput 또한 mixed-port, frontend, frequency, whole-loop 효과를 생략하므로, 이 model은 255H 우승자를 고르는 대신 공백을 기록하는 역할을 한다.

5.9 9파 encoding, compact block, semantic timing

이번 패스는 timing sample의 의미부터 강화했다. 이전 runner는 각 후보의 public 1라운드/20라운드 함수를 직접 검증하고 측정 loop를 audit했지만, 어느 gate도 main이 20라운드 함수를 선언된 횟수만큼 정확히 적용했는지는 증명하지 못했다. outer iteration을 절반만 실행하는 regression 소스가 두 gate를 모두 통과하면서 2.253배 빠른 것처럼 보였다. Schema 5는 독립적인 repeated-call oracle로 이 공백을 메운다. 모든 후보의 preflight, 폐기된 warm-up, 측정 process는 선언된 iteration count와 정확한 최종 256비트 state를 보고해야 한다. 따라서 3,000,000회 호출 campaign마다 후보당 39개 process(preflight 1, warm-up 6, sample 32)를 검증하고, 정확한 case set, count, state, canonical output hash를 fail-closed 기록에 저장한다. 음수/0 iteration, 중복되거나 누락된 field, hash/state 불일치, half-loop 소스는 regression test다. schema 2-4 JSON은 여전히 historical 성능/assembly 증거이지만 이 repeated-call 속성을 확립하지는 않는다.

이 micro-optimization은 3-operand VEX encoding의 commutativity를 이용한다. VPOR, VPXOR, VPADDQ의 두 source를 바꾸면 값과 의존관계는 그대로 유지하면서 번호가 낮은 변화하는 state를 더 짧은 ModRM 위치에 놓을 수 있다. 20개의 VPOR 각각이 1바이트씩 절약한다. 그 결과 정확한 GCC 13.3은 **122 instruction, 549바이트, hot memory 0개**를 낸다. 6-instruction 변환 하나가 27바이트이므로, 현재의 SUB-counter 형태는 encoding 하한 $20 \cdot 27 + 3 + 6 = 549$ 바이트에 도달한다. target-only인 use_incdec 대조군은 outer counter를 바꿔 548바이트에 도달한다. digest로 고정한 34-case 컴파일러/link 스크리닝은 모든 경우에서 표준 correctness gate와 정확한 audit을 통과했다. build들은 5개의 정규화된 stream과 9개의 stream/alignment class로 수렴하며, 모두 100.03/180.03 Alder/Zen-2 대리 경로를 유지한다.

독립적인 알고리즘 스크리닝은 one-pair loop와 complete unrolling 사이의 counted block들을 탐색했다. 채택된 block2는 몸체마다 2개의 round-pair를 처리하며 5번 반복한다. static timing 영역은 겨우 **136바이트, 30 instruction**이며, 모델링된 non-padding dynamic count는 122에서 133(alignment NOP 1개 포함 시 134)으로 늘어나고, hot memory는 여전히 0이고 같은 대리 critical path를 유지한다. block1은 75바이트/static 18개, 모델링된 non-padding dynamic instruction 143개(NOP 포함 144개)이고, block5는 321바이트/static 69개, 131개(NOP 포함 132개)이다. block5의 Alder reciprocal-throughput 대리 지표가 21.8로 block2의 22.2cycle보다 약간 낮지만, target 증거가 나오기 전에는 그 작은 model 이득이 훨씬 큰 static footprint를 정당화하지 못한다. MCA count는 alignment padding을 의도적으로 제외하며, 이는 속도 주장이 아니라 code-footprint trade-off다. 별도의 전수 스크리닝은 첫 두 scalar stage-major chain에 대해 $24 \cdot 24 = 576$ 가지 순서를 모두 시도했지만 조정된 120.06-cycle 대리 지표보다 엄밀히 나은 결과는 찾지 못했다.

이후 새 후보들을 CPU 1과 CPU 3에서 기존 569바이트 stream과, 위와 같은 warm-up/sample/call protocol을 schema 5 아래 사용해 비교했다.

Old569/candidate	CPU 1 paired (95% CI)	CPU 3 paired (95% CI)
549-byte commutative	1.000451 (0.999294–1.001894)	1.000466 (0.998164–1.002395)
548-byte inc/dec	0.999555 (0.997869–1.000627)	1.001880 (0.997620–1.004207)
549-byte aligned	0.998679 (0.994845–1.002219)	1.000329 (0.995564–1.003437)
Block2	0.999263 (0.997061–1.001998)	0.999278 (0.997635–1.002321)

모든 구간이 1을 포함한다. footprint상의 승리는 재현 가능하지만 target-only이며, 549/548바이트 형태와 block2는 적응형 scalar 제출 incumbent를 대체하지 못한다.

5.10 10파 counted front end, code generation, 강화된 timing

이전의 compact block은 intrinsic에서 나온 것이라 피할 수 있는 frontend instruction을 남겼다. 정확한 GCC 13.3 quotient/remainder 스크리닝은 전통적인 DEC/JNE 제어로 10가지 pair-block 크기를 모두 생성해 3개의 Pareto point를 남겼다.

AVX2 stream	Bytes	Static inst.	Modeled dynamic	Alder RThroughput
Full 20-transform inline	549	122	122	20.3
Block2 ×5	122	29	133	22.2
Block3 ×3 + tail1	238	53	129	21.5
Block5 ×2	292	65	127	21.2

모든 count는 clock으로 구획한 전체 loop를 대상으로 하며 dynamic 열은 alignment padding을 제외하고, 모든 stream은 hot memory operand가 0개이고 같은 100.03-cycle 모델링 dependency 경로를 갖는다. Block2는 기존 intrinsic block2의 136바이트/static 30 instruction보다 같은 133 dynamic instruction에서 엄밀히 더 낮다. Block3도 기존 intrinsic block5의 321바이트/static 69/dynamic 131 구조를 능가한다. block 6-10은 20개 변환을 모두 static하게 남긴 채 loop 제어만 추가하며, LOOP는 signed 8비트 범위를 초과하거나 Alder model에서 불리하게 확장된다. 생성된 13개 분해 전부가 정확한 binary audit과 표준 correctness gate를 통과했다.

별도의 29-case register/expression/boundary 스크리닝은 현재 6-instruction 변환의 549바이트 하한을 낮추지 못했다. high-state, low-scratch register 배정 3가지는 baseline과 정확히 동물이었다. allocator가 high scratch register를 고르게 하면 loop가 569바이트로 커졌고, high state를 ModRM에 두면 609바이트로 커졌으며, 모든 고정 register class를 과도하게 제약하면 563바이트, 124 instruction, hot constant reload가 생겼다. 4가지의 정확한 재결합은 122 instruction과 549바이트를 유지했다. constant XOR를 right-shift 분기로 옮기면 Zen-2 대리 지표만 180.03에서 160.03cycle로 줄었고, Alder Lake와 Meteor Lake는 100.03 그대로였으므로 Intel-target 승격은 아니다. 14가지의 통제된 시작 offset은 중요한 16/32/64바이트 경계를 모두 다루었지만 정규화된 stream은 하나뿐이었으며, static llvm-mca는 연결된 fetch-boundary 효과의 순위를 매길 수 없다. 모든 full build는 표준 correctness gate와 정확한 측정-loop audit을 통과했다.

scalar 크기 스크리닝도 교체 후보를 찾지 못했다. 같은 register를 강제한 ROL은 유일한 move를 없애 322 instruction을 유지하면서 loop를 1,211바이트에서 1,052바이트로 줄이지만, Alder/Meteor 대리 지표를 121.06cycle/402 uop에서 138.13cycle/482 uop로 늘린다. 같은 register를 쓰는 SHLD는 정확한 rotate이지만, 직렬 microprobe 비용이 RORX의 1.03 대비 3.03 모델 cycle이라, 80회 사용으로 확장하기 전에 기각되었다. 이는 code-size 관측일 뿐 target speedup이 아니다.

이후 3개의 counted 후보와 549바이트 full stream을 CPU 1과 CPU 3에서 위와 같은 warm-up/sample/call protocol로, schema 5로 강화해 측정했다. 표는 full549/candidate paired median과 결정론적 5,000-resample bootstrap 구간을 보고한다.

후보	CPU 1 paired (95% CI)	CPU 3 paired (95% CI)
Block2 ×5	1.002787 (0.993620-1.008628)	0.999423 (0.996616-1.002621)
Block3 ×3 + tail1	1.005277 (1.001490-1.014413)	1.001329 (0.997071-1.004260)
Block5 ×2	0.999420 (0.990163-1.008628)	1.000233 (0.997524-1.002615)

6개 구간 중 5개가 1을 포함한다. CPU 1의 유일한 block3 신호는 0.53%에 불과해 1% 승격 문턱보다 낮고, 그 실행은 눈에 띄게 비정상적이다. full stream의 처음 7개 sample은 36.07-37.44 ns인 반면 나머지 sample은 대부분 55-64 ns 범위로 뛰어오른다. CPU 3는 이 신호를 재현하지 못한다. 따라서 이는 승리가 아니라 진단용 증거이며, 측정된 binary들도 표준 correctness gate와 정확한 loop audit을 통과했다. 별도의 oracle이 모든 preflight, warm-up, 측정 process를 선언된 iteration count에서 검증했고, 새로 생성한 128비트 nonce, campaign id, 소스 hash, 측정 count가 독립적인 semantic challenge를 위한 두 번째 예측 불가능한 count를 유도했다.

세 가지 적대적 regression이 추가된 gate들이 다루는 범위를 규정한다. 호출의 절반만 timing하고 나머지 절반은 clock() 이후에 재계산하는 소스는 0.65–1.05 inner-time/child-CPU median coverage gate에서 실패한다. 호출의 절반만 timing하고 이후 알려진 최종 state를 그대로 쓰는 소스는 non-ce로 유도한 대체-count challenge에서 실패한다. 출력 평균을 그냥 절반으로 줄이기만 하는 소스는 total elapsed time과 선언된 count에 대한 정확한 일관성 검사에서 실패한다. 이 검사들은 기록된 sample을 테스트 대상 소스에 더 단단히 결속하지만, 생성된 두 build를 모두 인식하는 임의의 적대적 코드에 대한 attestation은 아니며, 특정 instruction이 iteration당 정확히 한 번 실행됨을 증명하지도 않는다. 승격에는 여전히 소스 검토, 정확한 clock-구획 assembly 계약, 차분 테스트, 독립적인 255H 측정이 필요하다.

결과적으로 10파의 어떤 결과도 적응형 scalar 제출 incumbent를 대체하지 못한다. 더 작은 block2, block3+tail1, block5 frontend는 실제 Core Ultra 7 255H에서 중요할 수 있는 target-only 대안으로 남지만, AMD timing도 static scheduling 대리 지표도 그 사이에서 하나를 선택해 주지는 못한다.

5.11 11파 stationarity gate와 제한된 ISA synthesis

앞서 본 CPU 1 실행의 phase 변화는 사후에 유리한 구간을 고르는 대신 고정된 promotion gate를 도입하도록 이끌었다. 이제 각 case는 최소 16개 sample이 필요하며, 4개의 미리 선언된 연속 block이 각각 완전한 균형 순서를 보존하도록 count는 case 수의 4배의 배수여야 한다. 절대적인 block-median drift는 5%를 넘을 수 없고, baseline/candidate 쌍의 block-median 비율 편차는 2%를 넘을 수 없으며, 한 block이 0.995 미만이고 다른 block이 1.005를 초과하면 그 쌍은 격리된다. 실패한 후보는 baseline 자체가 실패하지 않는 한 그 쌍만 격리한다. autotuner는 raw sample로부터 전체 기록을 독립적으로 재계산하며 바뀐 요약값이나 threshold는 거부한다. 이는 stationarity의 증명이 아니라 보수적인 promotion 방어벽이며, block 내부의 짧은 변동은 놓칠 수 있다. Barrett 등의 changepoint 연구와 Kalibera and Jones의 repetition/effect size 방법론은, warm-up이 steady state를 만들었다고 가정하는 대신 phase 거동과 불확실성을 1급 증거로 다루도록 동기를 부여한다. 이 fixed-block gate는 Barrett 등의 changepoint/PELT 분석을 재현한 것은 아니다.

새로 수행한 CPU 1과 CPU 3 측정은 6회의 폐기된 warm-up, 24개의 균형 sample, sample당 3,000,000회 호출을 사용했다. CPU 1의 full AVX2 baseline과 4개의 AVX2 대안 모두 절대/pairwise gate를 통과한다.

CPU 1 후보	Full549/candidate (95% CI)	Gate
Old intrinsic block2	0.999624 (0.997787–1.002890)	eligible
Counted block2	1.000750 (0.997649–1.006677)	eligible
Block3 + tail1	1.002591 (0.998829–1.006870)	eligible
Counted block5	1.002347 (0.999722–1.004974)	eligible

모든 구간이 1을 포함하고, eligible한 median 어느 것도 1.010 promotion threshold에 이르지 못한다. CPU 1의 scalar 절대 block median은 9.06%, paired 효과는 7.13% 흔들려서, 종합 비율이 크더라도 그 비교는 진단용에 불과하다. CPU 3에서는 full baseline이 8.28% 흔들리고, 모든 case가 5% 절대 한계를 넘으며 여러 AVX2 쌍이 $\pm 0.5\%$ 넘게 역전된다. 따라서 CPU 3 측정 전체가 진단용이다. 이 결과들은 두 stationarity_gate_timing JSON 파일에 raw sample과 함께 저장되어 있다.

알고리즘 패스는 BSWAP64(ROL_r(x))가 어떤 contest rotation에서도 단 3개의 AVX2 instruction만으로 표현될 수 있는지 물었다. 전수 열거와 구조적 배제를 결합해, qword 논리 shift, 임의의 고정된 word-local byte 선택, XOR/OR(서로소 field에 대한 carry-free ADD 포함)로 이루어진 제한된 grammar에 대해 3-instruction single-input DAG 모양 전부에서 UNSAT임을 확립했다. 더 정확히는, parallel-two-unary-plus-combine 모양은 전수 열거했고, bijectivity와 projection 논증이 다른 모양들을 구조적으로 배제한다. 이 열거+구조적 배제는 이 grammar에 대한 lower bound일 뿐, 모든 x86 opcode나 machine program에 대한 것은 아니다. 정확한 split-shuffle 대조군 3개는 표준 correctness gate와 binary audit을 통과했지만, incumbent의 122 instruction/549바이트에 비해 **142 instruction과 669–689바이트**가 필요하다. shuffle을 XOR 너머로 옮기는 것은 constant를 변환하면

가능하지만, 그 사이에 있는 modular ADD 너머로 옮기는 것은 불가능한데, byte를 넘나드는 carry가 명시적인 반례를 제공하기 때문이다. 따라서 두 round-경계 shuffle은 그런 재결합으로 상쇄되지 않는다.

LLVM 19도 하드웨어 공백을 메우지 못한다. 테스트한 모든 stream에서 alderlake, arrowlake, arrowlake-s, lunarlake, meteorlake, sierraforest가 동일한 cycle, uop, reciprocal throughput을 보고했다. 해당 release의 공식 X86.td는 노출된 arrowlake processor를 AlderlakePModel에 매핑하므로, 서로 다른 label이 독립적인 255H microarchitecture 증거가 되지는 못한다. llvm-mca 안내서도 경고하듯, 이는 측정이 아니라 static scheduling-model 추정치다.

유일하게 남은 promotion 경로는 방어된 probe--screen--confirm--decide workflow다. 이는 실제 Core Ultra 7 255H의 허용된 모든 logical CPU에 CPUID probe를 고정하고, core type을 Linux topology 신호와 결합하며, 모호한 LP-E 분류는 잠정 상태로 남긴다. screen 뒤에는 decide가 승격하기 전 두 번의 독립적인 확인 세션이 따라야 한다. 교체하려면 두 세션, 요청된 각 core type의 물리적 대표 2개, 일치하는 compiler/source/manifest hash, 직접 correctness, 정확한 측정-binary SHA-256/assembly gate가 필요하며, 재사용된 artifact 경로나 hash는 또 다른 세션으로 인정되지 않는다. 각 실행은 새 128비트 id를 자신의 index와 benchmark JSON에 결속하며, canonical 증거와 paired-sample hash는 공백만 바꿔 이름을 바꾼 복사본도 거부한다. canonical fingerprint는 추가로 autotuner와 benchmark driver, loop audit, 독립적인 oracle/verifier, 공식 archive, Python, objdump, size 실행 파일을 고정한다. 변경된 screen protocol이나 서로 다른 확인 protocol은 새 측정을 강제하며, 오래된 topology probe로는 새 screen을 시작할 수 없다. correctness metadata는 1라운드와 20라운드 모두에서 random state와 random ADD/XOR constant를 명시적으로 기록해야 하며, runner는 verifier의 정확한 count, seed, round, constant, PASS 출력을 파싱한다. reference translation unit은 고정된 중립 flag를 사용하고, 후보 flag는 그 object와 link에만 적용된다. 소스 바이트는 한 번 스냅샷되며 원본과 rewritten-성능 hash가 모두 보존된다. 중첩된 topology 기록은 각 cache-list 원소를 통해 검증되고, 잘못된 입력은 traceback 없이 fail-closed된다. Schema 5는 추가로 앞서 설명한 repeated-call oracle, 정확한 case set, 모든 timed-process 최종-state 기록을 요구한다. 확장된 균형 15-case integration 스크리닝은 15개 직접 correctness 검사와 15개 정확한 binary audit을 모두 통과했다. 그 1,000회 호출 timing은 tool-regression 증거일 뿐이다. 이전의 19-case manifest(partial-unroll 대조군과 lane-wise AVX2 포함)는 19개 검사와 audit을 모두 통과했다. 서로 다른 -fira-region=one과 single-scratch 569바이트 stream을 추가해 21-case smoke test를 만들었고 21/21 직접 검사와 audit을 통과했다. 7개의 non-baseline stream/alignment 대표를 추가해 28-case manifest를 만들었고, 새로 수행한 균형 smoke test는 28/28 직접 검사와 28/28 정확한 binary audit을 통과했으며 모든 case에 source-local -iquote context가 기록되었다. 그 1,000회 호출 provisional-host timing은 integration 증거일 뿐이다. 549바이트 commutative 소스가 예전 assembly 항목을 대체해, 그 548바이트 counter 대조군과 예전 intrinsic block2와 함께 30-case schema-5 manifest를 이루었다. 3개의 counted block2/block3/block5 frontend가 이를 33개 target case로 확장한다. 세 추가 항목 모두 각자의 전용 random-state/random-constant 및 정확한 audit 스크리닝을 통과했다. 의도적으로 축소된 AMD/GCC12 확인 세션 2개는, 누락된 모든 target/compiler/sample/warm-up/iteration/random-case gate를 명시하면서 incumbent를 유지했다. 모든 P-core 측정은 paired median ≥ 1.010 과 E/LP-E 회귀 없는 adjusted lower bound > 1.005 가 필요하다. 조건을 만족하는 후보가 여럿이더라도, 직접적인 head-to-head 비교 전까지는 의도적으로 incumbent를 유지한다.

6 벤치마크 결과

이전 microbenchmark는 outer iteration마다 NOINLINE 20라운드 함수를 호출했지만, GCC는 최적화 이전 제출본을 제공된 timing loop에 완전히 inline했다. 이 차이는 측정치를 몇 퍼센트 바꾸며, 관측된 한 실행에서 두 median을 따로 정렬해 나눈 비율은 paired sample 비율의 median과 약 4퍼센트 차이가 났다.

이 benchmark는 complete contest 파일을 각각 컴파일하고, 같은 소스의 두 build를 포함해 case별로 flag를 지정할 수 있다. 제공된 timing loop의 호출 횟수만 바꾸며, whole-process warm-up을 폐기하고(확인 campaign에서는 6회), 하나의 logical CPU에 고정하며, 순환/역순 case 순서를

사용한다. raw total clock(), 출력된 평균, child-CPU, 외부 wall sample, median, MAD, p05/p95, 결정론적 bootstrap median 구간, paired speedup, outlier flag를 모두 JSON에 보존한다. 보고되는 호출당 값은 total elapsed time으로부터 재계산되며, eligible한 score campaign은 타당한 inner-time/child-CPU coverage를 요구한다. 모든 process는 공식 벡터 검사도 다시 실행한다. timing 전에 schema 5는 각 contest.c를 main만 이름을 바꿔 컴파일하고, 독립적인 verifier translation unit과 link해 그 후보를 직접 호출한다. 이는 oracle self-test를 후보 차분 검증과 분리해 유지한다. --audit-mode가 지정된 모든 case에 대해서는, 어떤 warm-up이나 측정 실행 전에 정확한 성능 실행 파일을 그 timed-loop 계약에 대해 audit한다. 권장 명령들은 모든 case에 mode를 지정한다. 같은 JSON은 canonical한 측정-protocol fingerprint도 기록해, 나중의 결정이 runner, verifier, archive, Python, binary-tool 버전을 몰래 섞지 못하게 한다. verifier 전용 flag override가 필요한 후보는 승격되는 대신 격리된다. 독립적인 repeated-call oracle은 요청된 iteration count에 대한 기대 최종 state를 고정하며, 모든 preflight, warm-up, 측정 process는 그 state를 정확히 보고해야 한다. nonce로 유도한 대체-count build는 timing이 시작되기 전에 독립적으로 그 oracle state에 도달해야 한다.

10,000,000회 호출, 15-sample campaign은 다음을 냈다.

Contest 형태 구현	Median	MAD	p05-p95
이전 scalar	36.799 ns	0.924 ns	35.766-45.773 ns
2라운드 unroll + BMI2	34.669 ns	2.232 ns	32.384-46.657 ns

이 historical한 최적화 이전/최종 소스 비교는 median paired speedup 1.068x, MAD 0.045x, bootstrap 95% 구간 1.008x-1.105x를 냈다. 더 잡음이 많은 별도의 20,000,000회 호출 campaign은 1.019x와 구간 0.991x-1.082x를 냈다. 둘 다 보고하는 것은 유리한 공유-VM 실행만 선택하는 것을 피하기 위함이다.

결정적인 실험은 대신 최종 소스를 기본 flag와 -mbmi2 -finline-limit=2000 양쪽으로 컴파일했다. 독립적인 두 번의 3,000,000회 호출, 21-sample campaign은 다음을 냈다.

Affinity	Default	Inline	Paired	Bootstrap 95% CI
CPU 0	44.467 ns	39.859 ns	1.116x	1.110-1.143x
CPU 4	45.924 ns	40.801 ns	1.118x	1.095-1.140x

paired MAD 값은 0.0281x와 0.0322x였다. 정확한 GCC 13.3 검사는 inline-limit 700/2000의 동등성과 call 제거를 확립했다. 따라서 최종 Intel Core Ultra 7 255H 실험은 incumbent를 Alder 및 Alder+IRA schedule, source order 2, 1, 0, 3, shortlist에 오른 두 backend scheduler stream, lane-wise AVX2, 예전 569바이트 stream, 7개의 nonbaseline stream/alignment 대표와, 각 core type의 기록된 물리적 대표에 고정한 독립 세션에서의 549/548바이트 encoding 및 block2 후보와 비교한다. native tune은 여전히 진단용으로만 남는다.

7 재현 및 코드 검증

제출 패키지의 src 디렉터리(submissions/02/src)에서 다음을 실행한다.

```
python3 benchmark_permutation.py \
  --case default=contest.c \
  --case inline=contest.c --baseline default \
  --case-cflag inline=-mbmi2 \
  --case-cflag inline=-finline-limit=2000 \
  --audit-mode default=default-call-allowed \
  --audit-mode inline=full-inline-320 \
  --cpu auto --iterations 3000000 \
  --warmups 3 --samples 21 --random-cases 100000 \
```

```

--json /tmp/challenge-inline.json

python3 optimization/audit_inline.py \
--json /tmp/challenge-assembly.json
python3 optimization/analyze_transform_lower_bound.py
python3 optimization/analyze_constant_placement.py \
--emit-dir /tmp/ch2-constants/candidates \
--json /tmp/ch2-constants/analysis.json
python3 optimization/reproduce_gcc133.py \
--json /tmp/challenge-gcc133.json
python3 optimization/screen_gcc133_schedules.py \
--json /tmp/challenge-gcc133-schedule-screen.json
python3 optimization/screen_gcc133_source_orders.py \
--json /tmp/challenge-gcc133-source-orders.json
python3 optimization/screen_gcc133_layout.py \
--json /tmp/challenge-gcc133-layout.json
python3 optimization/analyze_two_round_superopt.py \
--json /tmp/challenge-superopt.json
python3 optimization/screen_simd.py \
--json /tmp/challenge-simd.json
python3 optimization/screen_255h_toolchains.py \
--json /tmp/challenge-toolchains.json
python3 optimization/screen_split_simd.py \
--output /tmp/challenge-split-simd.json
python3 optimization/screen_avx2_codegen.py \
--output /tmp/challenge-avx2-codegen.json
python3 optimization/screen_avx2_inline_asm_alloc.py \
--output /tmp/challenge-avx2-inline-asm.json
python3 optimization/screen_phase_staggered.py \
--output /tmp/challenge-phase-staggered.json
python3 optimization/screen_avx2_commutative_layout.py \
--check
python3 optimization/screen_avx2_pair_blocks.py \
--check --jobs 4
python3 optimization/\
screen_tenth_avx2_counted_frontend.py --check
python3 optimization/\
screen_tenth_codegen.py --check
python3 optimization/\
screen_tenth_codegen_scalar_rotate.py --check
python3 optimization/\
screen_eleventh_isa_synthesis.py --check
python3 optimization/screen_gcc133_avx_flags.py \
--json /tmp/challenge-gcc133-avx-flags.json
python3 optimization/analyze_255h_instruction_model.py \
--check optimization/instruction_model_255h.json
python3 benchmark_permutation.py \
--case current=optimization/contest_simd_avx2_lanewise.c \
--case inline_asm=optimization/contest_simd_avx2_inline_asm.c \
--case phase=optimization/contest_simd_avx2_phase_staggered.c \
--baseline current \
--case-cflag current=-mavx2 \
--case-cflag current=-DCH2_SIMD_INLINE \
--case-cflag current=-finline-limit=2000 \
--case-cflag inline_asm=-mavx2 \
--case-cflag inline_asm=-DCH2_SIMD_INLINE \
--case-cflag inline_asm=-finline-limit=2000 \
--case-cflag phase=-mavx2 --case-cflag phase=-mbmi2 \

```

```

--case-cflag phase=DCH2_SIMD_INLINE \
--case-cflag phase=finline-limit=2000 \
--audit-mode current=avx2-inline-lanewise \
--audit-mode inline_asm=avx2-inline-lanewise \
--audit-mode phase=report-only \
--cpu 1 --iterations 3000000 --warmups 6 --samples 32 \
--random-cases 100000 \
--json /tmp/challenge-eighth-cpu1.json
python3 optimization/benchmark_timing_stability.py \
--cpu 2 --iterations 3000000 --warmups 6 --samples 32 \
--random-cases 100000 --json /tmp/challenge-timing-stability.json
python3 optimization/autotune_255h.py probe \
--compiler /path/to/gcc-13.3.0 \
--out /tmp/ch2-255h/topology.json

```

세 개의 10파 --check 명령은 각자의 static screening 결과를 재생성해, checked-in 상태인 counted-frontend, code-generation, scalar-rotate 결과 JSON과 비교한다. 강화된 두 host campaign은 별도로 src/optimization/tenth_wave_timing_cpu1.json과 tenth_wave_timing_cpu3.json에 보존되어 있으며, 32개의 raw sample 쌍 전부와 source/protocol/binary hash, semantic challenge, child-CPU coverage 기록을 담고 있다. timing과 새로 생성한 nonce는 의도적으로 변동성이 있으므로, 이 파일들은 byte 단위로 재현 가능한 --check 결과물이 아니라 측정된 증거다. 정확한 6-case timing 호출은 submissions/02/README.md에 기록되어 있다. 11파 --check는 마찬가지로 eleventh_isa_synthesis_results.json을 검증한다. 그 두 stationarity campaign은 stationarity_gate_timing_cpu1.json과 stationarity_gate_timing_cpu3.json으로 보존되어 있으며, gate 요약뿐 아니라 raw sample 자체가 결정 근거다.

더 긴 후보 행렬에 대해서는 다음을 실행한다.

```

python3 optimization/run_benchmarks.py \
--profile native --iterations 7000000 \
--warmup-iterations 3000000 --repeats 21 \
--random-cases 100000

```

18-후보 frontend/cache 스왑은 다음으로 재현할 수 있다.

```

python3 optimization/run_deep_review.py \
--iterations 2000000 --warmup-iterations 300000 \
--samples 15 --random-cases 100000

```

제출된 소스는 제공된 호환성 flag를 지원한다. clean-checkout wrapper는 원본 archive에서 공식 벡터를 임시 디렉터리로 추출하고, 권장 score build를 실행한 뒤, 생성된 모든 파일을 제거한다. 이 확인만은 다른 명령들과 달리 repository root에서 실행한다.

```

gcc -O3 -Wall -Wextra -o /tmp/challenge-default \
submissions/02/src/contest.c
./submissions/02/src/run_contest.sh

```

8 참고 자료

- ISO/IEC JTC1/SC22/WG14, [N1570: C11 committee draft](#): exact-width integers, unsigned modular arithmetic, and valid shifts.
- GCC, [Byte-Swapping Builtins](#): the fixed reverse-map specialization.
- GCC, [Common Function Attributes](#): semantics and linker limitations of aligned, noline, noclone, and always_inline.
- GCC, [x86 Function Attributes](#): local BMI2 code generation under the supplied build command.
- GCC, [GCC 13.3 Optimize Options](#): inline thresholds, scheduling, IRA, PGO, LTO, and loop/jump/label-alignment controls.

- de Moura and Bjørner, [Z3: An Efficient SMT Solver](#), TACAS 2008, and the [official Z3 publication list](#): bit-vector equivalence and UNSAT checks for the scalar operation-deletion search.
- Solar-Lezama et al., [Combinatorial Sketching for Finite Programs](#), ASPLOS 2006: the CEGIS-style template-and-counterexample synthesis procedure.
- LLVM, [llvm-mca Machine Code Analyzer](#): scheduling-model throughput and resource analysis for the LLVM 16/19 proxy screens, including the model-accuracy limitation of static estimates.
- LLVM, [LLVM 19.1.7 X86 processor-model definitions](#): the official source mapping the exposed arrowlake processor name to AlderLakePModel.
- Abel and Reineke, [uops.info: Characterizing Latency, Throughput, and Port Usage of Instructions on Intel Microarchitectures](#), ASPLOS 2019: separating instruction count from latency, throughput, and execution-port behavior.
- Abel and Reineke, [nanoBench: A Low-Overhead Tool for Running Microbenchmarks on x86 Systems](#): warm-up, serialization, repetition, and counter-aware low-level measurement design. Required counters were unavailable on this VM.
- Barrett, Bolz-Tereick, Killick, Mount, and Tratt, [Virtual Machine Warmup Blows Hot and Cold](#), OOPSLA 2017: changepoint evidence that discarded warm-up alone does not guarantee a steady state. The present native-code gate uses this as methodological motivation, not as a claim that its VM/JIT model applies directly.
- Kalibera and Jones, [Rigorous Benchmarking in Reasonable Time](#), ISMM 2013: planned repetition, uncertainty attribution, and effect-size confidence intervals.
- Intel, [Intel 64 and IA-32 Optimization Reference Manual](#), and the [Intrinsics Guide](#): scalar/BMI2 and AVX2 candidate analysis.
- Intel, [Intel 64 and IA-32 Architectures Software Developer Manuals](#): VEX.vvvv, ModRM r/m, extension-bit, and instruction-length interpretation for the commutative operand-order/register-placement screen, plus the RORX, ROL, and SHLD semantics and encodings used by the scalar-rotate screen.
- Intel, [Skymont Instruction Throughput and Latency](#): a Xeon 6 E-core-scoped package whose transfer to client 255H remains conditional.
- Intel, [Crestmont and Redwood Cove Instruction Throughput and Latency](#): conditional LP-E rows if the PerfMon mapping describes the 255H execution core.
- Intel, [Arrow Lake Performance Monitoring Events](#): Lion Cove P, Skymont E, and Crestmont LP-E mapping, one side of the official-source conflict.
- Intel, [Heterogeneous Computing on Intel Core Ultra 7 255H](#): calls the two LP-E cores additional Skymont cores, conflicting with the PerfMon Crestmont label.
- GCC, [GCC 13.3 x86 Options](#): AVX2 target and tuning controls, Alder Lake tuning availability, and why AMD `-march=native` data cannot stand in for the Intel target.
- Intel, [Core Ultra 7 255H specifications](#): 6 P, 8 E, and 2 LP-E cores and AVX2 as the listed vector extension.
- Intel, [Game Dev Guide for 12th Gen Intel Core Processor](#): hybrid CUID flag, per-logical-CPU leaf 0x1a core type, and affinity-pinned probing.
- Linux kernel, [CPUSETS documentation](#): affinity isolation and the system-level basis for recording pinned benchmark CPUs.
- Linux kernel, [CPU topology documentation](#) and the [CPU sysfs ABI](#): package/core/thread sibling and capacity cross-checks.
- Mytkowicz et al., [Producing Wrong Data Without Doing Anything Obviously Wrong!](#): code-layout and execution-order benchmark bias.
- Claude Carlet, [Boolean Functions for Cryptography and Coding Theory](#): ANF degree and finite-difference background for the table-separability checks.
- Google, [Benchmark User Guide](#): warm-up, repetitions, interleaving, statistics, and machine-readable output.
- NIST, [Measures of Scale](#): MAD as a robust dispersion report for noisy samples.